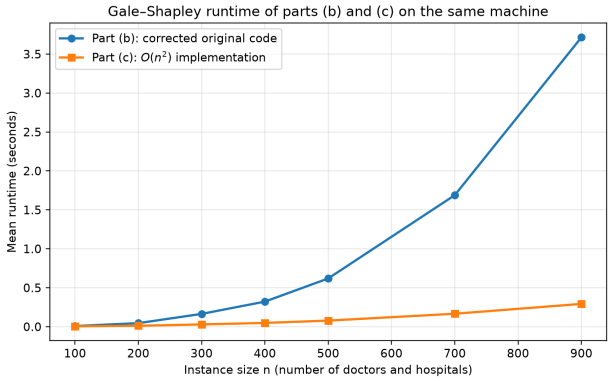
CS 5112 Fall 2025	HW1: *Stable Matching, Greedy, Divide and Conquer*
Name: Anirudha Hadagali	**Due: September 16, 11:59 PM ET**
NetID: ah2495	
Collaborators: Fill Here	



n	Part (b)	Part (c)	(b)/(c)
100	0.008 s	0.005 s	1.5×
300	0.164 s	0.029 s	5.6×
500	0.619 s	0.078 s	7.9×
700	1.689 s	0.166 s	10.1×
900	3.714 s	0.293 s	12.7×

Problem 1.

(a) (*2 points*) ()

Answer: *None*

(b) (*6 points*) ()

Answer: It does not achieve the O(n ** 2) time complexity because there are the two loops nested, but also some expensive operations inside each iteration. When we pop from a list, that costs O(n) because you need to move all the other elements up one position to fill the gap left by the first element that gets removed. The code later does a linear scan through the scan looking for a matching value in a list, which is another O(n) operation.

(c) (*8 points*) ()

Answer: In part (b), Gale–Shapley is correct but not O(n**2). Each proposal does two O(n) operations: pop(0) to get the next doctor, and list.index to compare hospitals on a doctor's list. Building a rank table is one of the most important optimizations to make, as each preference only takes O(1), rather than O(n). Another improvement is not using .pop(0) when attempting to find the next doctor. Instead, we can use a next doctor pointer with O(1) time.

I called (b) method and (c) method on the same file and averaged 3 trials. At n=900, (b) took 3.71

seconds and (c) took 0.29 seconds, about 13× faster. As n grows, (b) pulls away, which is what we expect.

I have attached the table (it keeps formatting to the top of the file, sorry).

(d) (*4 points*) ()

Answer: It is worst-case because Gale–Shapley makes 7 proposals, and for n = 3 that is the maximum possible: n**2 − n + 1 = 7

Each hospital can propose to each doctor at most once, so there are at most n**2 = 9 proposals. The algorithm always finishes with a perfect matching, so it cannot use all 9; the last hospital that gets matched never needs to go all the way down its list.

The lists are set up so almost every engagement gets broken before the last doctor is taken. Doctor 1 is everyone's last choice, so nobody proposes to D1 until the other options are exhausted

Problem 2.

(a) (*4 points*) ()

Answer: Incorrect

M = 5

I = [[0, 2], [2, 5], [1, 3], [3, 5]]

Optimal solution here is 2 intervals [[0, 2], [2, 5]]

The algorithm takes 3.

- J is empty, so c = 0. The only interval containing 0 is [0,2], which it must pick
- now [0,2] is covered, so the next uncovered point is starting at 2. Intervals that contain that point are [1, 3], [2, 5], [3, 5]. Earliest start is [1, 3].
- Now [0,3] is covered, remaining intervals are [2,5] and [3,5]. the earliest start is [2,5]

(b) (*4 points*) ()

Answer: Incorrect

M = 6

I = [[1, 5], [0, 3], [3, 6], [0, 2], [4, 6]]

Optimal solution here is 2 intervals [[0, 3], [3, 6]]

- J starts empty, and picking the longest interval is [1,5].
- Next, picking the next longest interval not already in coverage gives us either [0,3] or [3,6].
- meaning there is still some coverage left, forcing us to pick another interval.

Which causes us to have 3 intervals.

(c) (*4 points*) ()

Answer: Correct algorithm.

(d) (*4 points*) ()

Answer: M = 3 I = [[0, 1], [1, 2], [2, 3], [0, 3]]

Optimal solution here is 1 interval [[0, 3]]

- We start with J = I
- Every interval is redundant, so picking the latest finish time leads to deleting [0,3]
- Now left with the remaining intervals, none of which are redundant. Meaning you don't delete anymore, leaving you with 3 intervals

(e) (*4 points*) ()

Answer: I was not able to figure out how to complete the 'delete earliest redundant interval' in the required O(nlogn) time. I think the improvement that I can do is not to recompute the remaining redundant intervals after every iteration. This current implementation is very slow with O(n**4) worst time.

Here is the algorithm that I designed:

- Copy the input into J.
- For each interval, build the list of all other intervals and ask: is this one already covered by them?
- If several are redundant, remember the one with the smallest finish time.
- Delete that one and repeat.
- Stop when nothing is redundant.

Problem 3.

(a) (*8 points*) ()

Answer: *None*

(b) (*12 points*) ()

Answer: *None*

Problem 4.

(a) (*7 points*) ()

Answer: This question is very similar to a LeetCode question regarding TwoSum. You basically want to double loop through the array and check each number with another. This costs O(n**2) because you are looping on the array twice. On the outer loop you hold onto a number, then you check with every other number using the inner loop, and you subtract the numbers to reach the target. If the target gets reached, you increase a global counter variable.

(b) (*4 points*) ()

Answer: Usually, when I see the O(n logn) time complexity needed, I immediately think of sorting the list. And in this case, I think it could be very valuable. Once we sort the list, we can calculate a new target by subtracting the d-mode from the current value when iterating. Because a(i) – a(j) = d-mode is the same as a(j) = a(i) – d-mode. We then use binary search to find the first occurrence of target in the values list. We also get the binary search to get the last index of the occurrence in the list. Once we know the range of first to last, we now know how frequent the target is. We add this range of occurrences to the global counter. We have to make sure we also account for counting the same index for target and a(i), thus subtracting one from the frequency.

(c) (*4 points*) ()

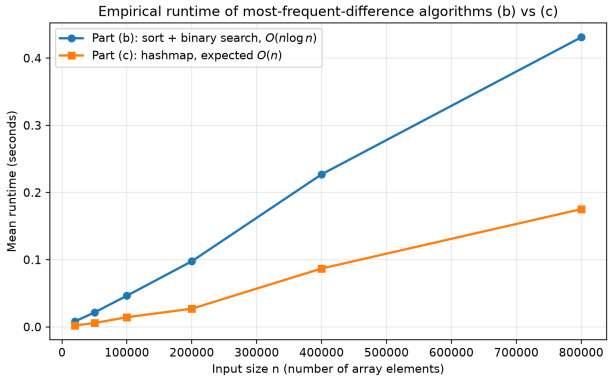
Answer: The best way to optimize the O(n**2) solution is to memoize the input. One way to do that is by creating a hashmap of all the possible values, along with their frequency. This lets you know which values are present, along with how many exist. Then you can loop through the list itself and calculate the new target you want to find. Once you know what you are looking for (the target), you can then look through your hashmap to see if it exists. If it does, then you add its frequency into counter. One thing that I had to check was making sure i != j, meaning not including the same index, which is a constraint in the problem.

(d) (*5 points*) ()

Answer: The O(n) approach uses linear time complexity by building the hashmap, iterating over values, and look up using hashmap. The hashmap itself is linear extra space.

(c) is faster; the trade-off is memory. Part (b) sorts and binary-searches, so it is O(n logn) time and little extra space. Part (c) uses a frequency hashmap, so it is expected O(n) time but Theta(n) extra space.

n	Part (b)	Part (c)	(b)/(c)
20,000	0.008 s	0.002 s	3.8×
100,000	0.046 s	0.014 s	3.2×
400,000	0.227 s	0.087 s	2.6×
800,000	0.431 s	0.175 s	2.5×



So: (c) buys an O(n) time vs O(n logn) time win by spending linear extra memory. (b) is slower but stays close to in-place. If memory is tight, or you must not copy the array into a table, (b) is the safer choice. If the array is large and you have RAM, (c) is better.

Problem 5.

(a) (*20 points*) ()

Answer:

1. Algorithm Design:

- Brute force: So, you calculate the starting point, and then from the start, you loop through all the apartments, calculate the distance from the starting spot, find the minimum distance, and add that as a tuple to the result set. And then you also store another array, which is all the previously visited apartments. And then for the next apartment in the visited list, calculate going to every remaining apartment, either from the starting apartment or from an apartment in the previously visited list. This approach is O(n**3), which is much worse that the desired optimal time, but this was a good starting point for me to understand what needed to be done.
- Now for the optimized O(n**2 logn) approach, I knew that I had to do some level of caching so that I could remove one of the loops from before. The initial idea I came up with was using a hashmap to store all the current best distances for a particular apartment. But this alone is not enough, because I need to know what the next shortest distance is and the corresponding path. For this, I had to implement a min heap, which was a new data structure for me. This min heap is necessary as it gives me the next min distance very quickly, without me having to calculate where to insert.
- For the actual algorithm, I store 4 different datasets. The results, visited apartments, heap, and hashmap. The results is the array that I use to store the actual 'handoffs' of the cookies. The visited apartments is a simple list of all the apartments that any of the friends visit to handoff cookies. The heap is to store the priority of which distance is lowest. The hashmap is for identifying the current shortest path for each apartment. Initially, I go through the entire list of apartments and populate the heap and hashmap. I do this by taking the starting point (Eman's apartment) and calculating the Manhattan distance to all her friends. Once this is calculated, I insert it into the hashmap with the 'key' being the friends' apartments, 'value' being the calculated distance and starting point. The same data is also stored in the heap with the distance being the 'key', and 'value' being a pair of apartments that create that path.
- I then have a while loop that runs until we have visited all the friends in the list. This allows us to keep searching for the shortest possible combination of apartments. Once inside the while loop, we pop the first element from the heap, which is designed to be the shortest available distance to any apartment. We make sure that the destination apartment is not already visited. Knowing that it is the shortest distance and we have not visited the apartment yet, we can conclude that this pair (Eman's apartment to destination) needs to go in the results list. We then loop through all possible apartments and calculate the Manhattan distance from this destination. The next part of the algorithm is determining if

this new distance calculated is better or worse than the previous distance calculated to this destination. In short terms, we go to the shortest path to a new apartment, then use that new apartment as the starting point, find the next 'possible' apartment from that starting point, and figure out if this new path is shorter or longer than the previous path to that end apartment. If it is shorter, we update the hashmap and heap accordingly.

1. Time complexity:

- The time complexity of my algorithm is O(n**2 logn). The O(n**2) comes from the double loop that I have, one being the outer while loop, and then the inner for loop. The O(logn) originates from pushing and popping from the heap via the heapq import in Python. Adding or removing an item restores the tree structure by either moving the elements upward or down.